Technical Report

Project: Host-Based Intrusion Detection (HIDS) + SOC AI Log Chatbox

Platform: Kali Linux (Local Host Deployment)

Author: Irene Omonefe Ovierheya Bazunu

Written For: ProStackHub

Date: October 2026

Table of Contents

1.0 Executive Summary
2.0 Aim and Objectives
2.1 Aim
2.2 Objectives
3.0 System Architecture and Design
3.1 Architecture Workflow Diagram
3.2 Directory and File Structure
4.0 Core Implementation
4.1 SQLite Database Layer (database.py)
4.2 Log Ingestion Engine (watcher.py)
4.3 AI Security Analyst (ai_agent.py)
4.4 SOC Interactive Dashboard (app.py)
5.0 Attack Simulation and Verification
5.1 SSH Brute-Force Testing with THC-Hydra
5.2 Privilege Escalation Verification
5.3 AI Copilot Incident Triage
5.4 Observation and Results
6.0 Technical Challenges and Solutions
6.1 Best Practices and Future Enhancements
7.0 Conclusion

1.0 Executive Summary

Organizations rely on servers that face continuous, automated cyber attacks from the Internet. Monitoring these servers manually by reading raw text logs is slow, tiring, and prone to human error.

This project delivers an automated Host-Based Intrusion Detection System (HIDS) paired with an AI Security Copilot. The system continuously monitors the host's authentication log (/var/log/auth.log), extracts high-risk security events such as SSH password guessing and unauthorized sudo attempts, records them in a fast SQLite database, and displays them on a clean Streamlit dashboard. A built-in AI analyst powered by Groq reads these logs on demand, summarizes threats in plain English, and provides direct firewall commands to stop attackers.

2.0 Aim and Objectives

2.1 Aim

To build, deploy, and evaluate an automated Host-Based Intrusion Detection System (HIDS) that captures host authentication logs in real time and uses a Large Language Model (LLM) to assist Security Operations Center (SOC) analysts in investigating incidents.

2.2 Objectives

Continuous Log Monitoring: Build a background daemon in Python to monitor /var/log/auth.log without consuming excessive CPU or RAM.

Intrusion Detection: Detect common brute-force attacks and privilege escalation patterns using regular expressions.

Structured Data Storage: Store security alerts in an indexed SQLite database table with timestamps, attack types, severity ratings, and source IP addresses.

Visual SOC Console: Develop a web dashboard using Streamlit to display metric cards, real-time alert logs, and severity breakdowns.

AI Incident Analysis: Integrate the Groq API (llama-3.1-8b-instant) to read recent security logs and answer security questions.

Realistic Attack Validation: Verify system accuracy by attacking the host using THC-Hydra and checking if alerts trigger immediately.

3.0 System Architecture and Design

3.1 Architecture Workflow Diagram



3.2 Directory and File Structure

task1_HIDS_AIchatbox/

```
├── .env                # Stores the secret GROQ_API_KEY
├── .gitignore          # Keeps credentials and virtual environments private
├── ai_agent.py         # Sends logs and user queries to the Groq API
├── app.py              # Streamlit web user interface
├── database.py         # Creates the SQLite table and handles database queries
├── requirements.txt    # List of required Python packages
├── watcher.py          # Watches auth.log in the background and flags threats
└── README.md           # Project documentation and setup instructions
```

4.0 Core Implementation

Phase 1: Environment Setup & Project Isolation

1. Kali terminal was opened to create a project directory called task1_HBID_AIchatbox inside PROSTACKHUB_PROJECTS directory which is the directory that contains my projects:

mkdir -p /home/kali/Documents/PROSTACKHUB_PROJECTS/task1_HBID_Aichatbox

2. task1_HBID_Aichatbox directory was navigated to using cd:

cd /home/kali/Documents/PROSTACKHUB_PROJECTS/task1_HIDS

3. A python isolated environment was created to prevent any dependencies installed from conflicting with the host dependencies library eg anaconda:

python3 -m venv venv

4. The environment was activated:

source venv/bin/activate

5. pip was installed and upgraded

pip install --upgrade pip

6. Dependencies were installed, watchdog, streamlit, groq, pandas

pip install watchdog streamlit groq pandas

7. A free API key was generated from console.groq.com and exported to the current environment:

export GROQ_API_KEY="gsk_your_actual_groq_api_key_here"

4.1 SQLite Database Layer (database.py)

Function: Captured alerts are saved into hids_events.db. The table schema organizes raw log data into clean, searchable columns:

A database.py was created to store flagged security events with timestamps, source, attack classifications, and severity tags This is to match regex patterns against log lines for failed SSH logins, brute-force patterns, and privilege escalation (sudo commands/root sessions). Type the bash:

nano database.py

4.2 Log Ingestion Engine (watcher.py)

A watcher.py was created using Watchdog to tail changes to syslog.log line-by-line as they occur and automatically trigger detector.py and database.py.

nano watcher.py

Function: The detection engine runs continuously with administrative privileges (sudo). It reads incoming entries appended to /var/log/auth.log.

The system uses regular expression filters to identify two primary threats:

a. SSH Brute-Force Logins: Matches failed authentication attempts (Failed password for invalid user ... from <IP>).

b. Unauthorized Privilege Escalation: Matches failed attempts to invoke sudo credentials or direct root escalation.

4.3 AI Security Analyst (ai_agent.py)

An ai_agent.py was created to retrieve recent events from SQLite and format them into an authoritative prompt for the ALLAM model on Groq.

nano ai_agent.py

Function: The AI agent uses the Groq API running the high-speed allam-2.7b model.

It securely loads credentials from .env using python-dotenv.

When triggered, it reads the 20 most recent alerts from database.py.

It formats the events into an analyst prompt, requesting threat summaries, attacker IP lists, and remediation steps.

4.4 SOC Interactive Dashboard (app.py)

Web application combining live alert metrics and the interactive Groq chatbot was created.

nano app.py

Function: Built using Streamlit, the dashboard provides a single screen for monitoring:

Metric Cards: Total alert count, number of unique attacker IPs, and high-severity counter.

Telemetry Table: A chronological table showing captured alerts.

AI Copilot Chat: An interactive chat box allowing the analyst to ask natural questions like "What happened in the last hour?" and get instant security recommendations.

5.0 Attack Simulation and Verification

Step 1: Operational Execution

Three terminal were opened on Kali Linux, and the following operations were carried out:

On Terminal 1:

1: Project directory was navigated into using the cd command

cd /home/kali/Documents/PROSTACKHUB_PROJECTS/task1_HID_AIchatbox

2. For hydra to generate real log entries on Kali, the SSH service must be running, hence, sudo systemctl was used to start, enable, and check the status of ssh already installed:

OpenSSH server was started:

sudo systemctl start ssh

Enabled ssh

sudo systemctl enable ssh

Verified it is active and listening on port 22:

sudo systemctl status ssh

3. I ensureg rsyslog was Active & Generating /var/log/syslog

Note: On Kali/Debian systems, /var/log/syslog is managed by rsyslog.

rsyslog daemon was checked if active

sudo systemctl status rsyslog

Note: If not installed or running, install and start it:

sudo apt update && sudo apt install rsyslog -y

After installation, the daemon was started, enabled, and the status was checked:

sudo systemctl start rsyslog

sudo systemctl enable rsyslog

sudo systemctl status rsyslog

4. The following bash was run to make watchdog send log to syslog.log folder in project:

sudo stdbuf -oL journalctl -f -n 0 -o short > syslog.log 2>&1 &

5. A bash command was run to trigger a log line so as to check if watchdog log is received by syslog.log:

sudo -k && sudo whoami

6. syslog.log file was checked to see if log was registered:

cat syslog.log

On Terminal 2 (Watcher Daemon Launch):

1. The project folder was navigated into using the cd command:

cd /home/kali/Documents/PROSTACKHUB_PROJECTS/task1_HBID_task1_AIchatbox

2. Python environment was created:

python3 -m venv venv

3. The environment was activated and navigated into:

source venv/bin/activate

4. API key from Groq was exported into the environment:

export GROQ_API_KEY="gsk_your_key_here"

5. Watcher Daemon was launched:

python3 watcher.py

On Terminal 3 (Streamlit Dashboard Launch):

1. The project folder was navigated into using the cd command:

cd /home/kali/Documents/PROSTACKHUB_PROJECTS/task1_HIDS

2. Python environment was created:

python3 -m venv venv

3. The environment was activated and navigated into:

source venv/bin/activate

4. API key from Groq was exported into the environment:

export GROQ_API_KEY="gsk_your_key_here"

5. OpenAI was installed through the venv environment terminal:

python -m pip install openai

6. The model available for the groq API key plan was checked. The type of groq account determines the AI model:

./venv/bin/python3 -c "from groq import Groq; import os; from dotenv import load_dotenv; load_dotenv(override=True); client = Groq(api_key=os.getenv('GROQ_API_KEY')); print([m.id for m in client.models.list().data])"

A list of models were shown.

7. allam-2.7b AI selected from the list of model, copied and pasted in ai_agent.py model section

nano ai_agent.py

Press Ctrl O and Enter, then Ctrl X to Save it.

8. Streamlit was then launched:

python3 -m streamlit run app.py

OR

use this command if anaconda or other library is conflicting with the venv environment:

/home/Kali/Documents/PROSTACKHUB_PROJECTS/Task1_HBID_AIchatbox/venv/bin/python3 -m streamlit run app.py

Step 2: Attack Simulation Validation (Terminal 1)

In terminal 1, SSH brute-force and privilege escalation simulations were initiated by feeding standard authentication entries into syslog.log:

5.1 A. SSH Brute-force:

1. A file was created in one single command. This was to create a brute-force attack scenerio:

printf "wrongpass1\ntoor\nadmin123\nletmein2026\n" > /tmp/passwords.txt

2. The contents of the files were verified using cat command:

cat /tmp/passwords.txt

Expected output:

All 4 passwords were listed on separate lines:

wrongpass1

toor

admin123

letmein2026

3. After verification, Hydra was run:

hydra -l invaliduser -P /tmp/passwords.txt ssh://127.0.0.1 -t 4

4. Watchdog Listener was checked on terminal 2 where watcher.py is running, to see live alert lines outputting:

[!] [ALERT] High | SSH Brute-Force / Failed Password | IP: 127.0.0.1…

5. syslog.log in the project file and streamlit HIDS + AI Chatbot dashoard were also viewed for brute-force login logs.

5.2 Execute Simulate Privilege Escalation

Privilege escalation events were triggered:

a. Trigger Failed Sudo / Command Abuse:

sudo -k && sudo whoami

b. Password request prompt was demanded to proceed. A wrong password was intentionally entered or typed, and Enter button was pressed.

c. Waited until it said sudo: 1 incorrect password attempt.

d. The command below was quickly run to trigger open root session:

sudo su

e. Kali password was entered to enter the root prompt (root@kali:#).

f. The command below was Immediately entered to exit back to user account:

exit

5.3 Streamlit & Ask the AI Copilot Inspection

a. Streamit would have automatically opened streamlit dashboard when launched. Alternatively, streamlit dashboard was opened by typing http://localhost:8501 in browser tab.

b. Refresh Telemetry on the left column of the dashboard was clicked.

c. Hydra failed attempts and root session events appeared in the table.

d. In the AI Chatbot on the right, this prompt was entered to check the AI functionality:

"What happened in the last hour? Are there any critical threats?"

5.4 Observation and Results:

a. SSH Brute-force:

watcher.py identified the failed attempts in real time.

It extracted the IP address 127.0.0.1 and assigned a HIGH severity rating.

The events were saved in hids_events.db within seconds.syslog.log showed [ALERT] High | SSH Brute-Force and [ALERT] Critical | Root Session Opened.

b. Privilege Escalation Verification

The engine detected the PAM authentication failure.

The event was categorized as FAILED_SUDO with a MEDIUM severity rating and displayed on the dashboard.

In Streamlit dashboard, all events were cataloged in SQLite.

c. A security analyst asked the AI Copilot:

"What happened in the last hour? Are there any critical threats?"

AI Copilot Output:

Correctly reported an active SSH brute-force attack against account invaliduser.
Listed the offending IP address (127.0.0.1).
Provided copy-paste defensive firewall commands:

sudo ufw deny from 127.0.0.1

sudo systemctl restart ssh

In the AI Chatbot input box, the typed prompt generated a response from the AI model.

6.0 Technical Challenges and Solutions

a. Python Shell and Environment Conflicts:

Problem: The global Anaconda (base) environment intercepted pip and python commands, leading to ModuleNotFoundError for groq and python-dotenv.

Solution: Deactivated Anaconda (conda deactivate) and used the direct virtual environment binary (./venv/bin/python3).

b. Environment Variable Formatting in .env:

Problem: A space after the equal sign (GROQ_API_KEY= gsk_...) caused the parser to load an invalid key or empty string.

Solution: Removed leading whitespace and configured load_dotenv(override=True) to force correct environment loading.

c. Model Identifier Updates (404 Error):

Problem: Model names like llama-3.3-70b-versatile returned 404: model not found due to tier limitations. This prevented real-time AI log analysis/

Solution: Updated the configuration in ai_agent.py to use the standard allam-2.7b model. And typed the prompt in the chatbox for AI security to read the available logs.

6.1 Best Practices and Future Enhancements

Automated Active Defense: Configure automatic IP bans using fail2ban or Linux nftables when an IP triggers more than 5 failed attempts in 60 seconds.

Log Rotation Support: Update watcher.py to watch file system inodes so it continues reading smoothly when Linux runs logrotate.

Credential Hygiene: Keep .env included in .gitignore so private API keys are never published to public Git repositories.

7.0. Conclusion

Host-Based Intrusion Detection (HIDS) + SOC AI Log Chatbox successfully delivered a functioning, automated Host-Based Intrusion Detection System. By combining low-level log file monitoring, a clean SQLite storage layer, an intuitive Streamlit interface, and high-speed Groq AI reasoning, the platform demonstrates how modern SOC operations can automate repetitive log analysis and accelerate incident response.